

PM_Collegio v2

Documentazione Tecnica — sistema gestione collegi universitari (refactor MVC + DAO + DTO)

Materia : Ingegneria del Software

Ateneo : Università di Pavia

Autore : Giresse Kengne

1. Introduzione

PM_Collegio è un sistema desktop sviluppato in Java (Swing/JFrame) per la gestione delle prenotazioni di camere in collegi universitari. L'applicazione supporta uno scenario multi-committente ma nella versione attuale è ottimizzata per un singolo committente. Il database di supporto è MySQL (schema pm_collegio). L'applicazione offre un'interfaccia grafica completa che guida l'operatore attraverso tutte le fasi operative: registrazione degli ospiti, assegnazione delle camere, check-in e check-out, generazione e gestione delle fatture, e supervisione amministrativa con controllo degli accessi basato su ruolo. La versione 2 rappresenta il refactor della versione 1 (monolitica) verso una architettura a strati MVC + DAO + DTO.

1.1 Obiettivi del refactor

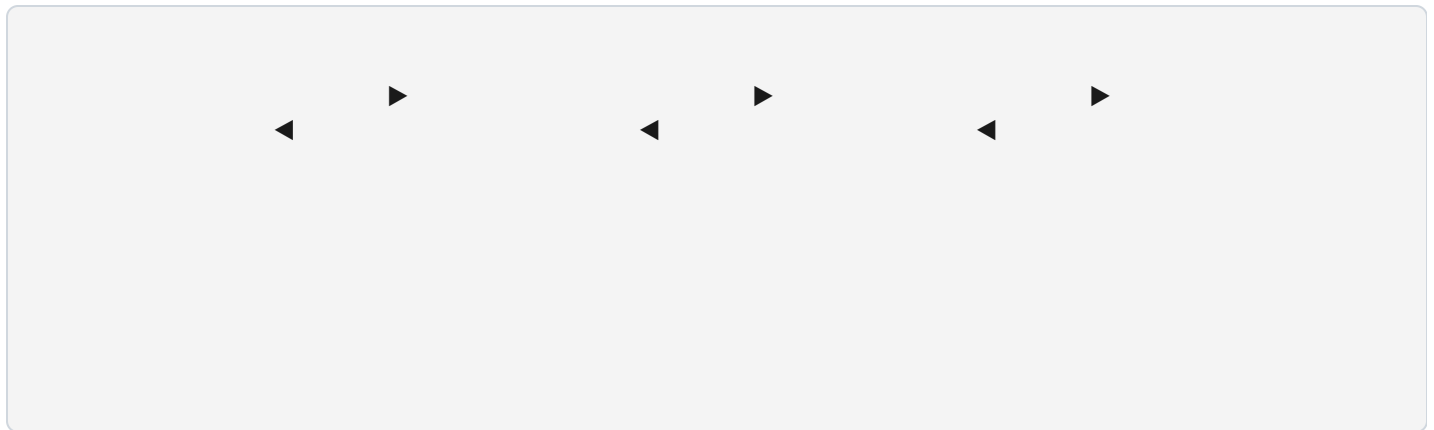
- Separare nettamente UI, logica applicativa e accesso ai dati
- Centralizzare le query SQL in un unico punto (`QueryContainer`)
- Eliminare la duplicazione di codice di connessione DB sparso in ogni view
- Introdurre transazioni esplicite per operazioni atomiche (CheckIn, CheckOut, audit)
- Applicare validazioni e calcoli *server-side* (controller) invece che inline nelle view

1.2 Ruoli supportati

Codice	Ruolo	Permessi principali
AS	Admin sistema	Tutto, inclusa gestione committenti
AC	Admin committente	Gestione del proprio collegio (camere, utenti, prenotazioni)
AR	Receptionist	Check-in / check-out, gestione prenotazioni
U	Cliente	Vista limitata : solo proprie prenotazioni e fatture, profilo modificabile

2. Architettura

Pattern : MVC (Model-View-Controller) esteso con DAO (Data Access Object) e DTO (Data Transfer Object).



2.1 Regole architetturali

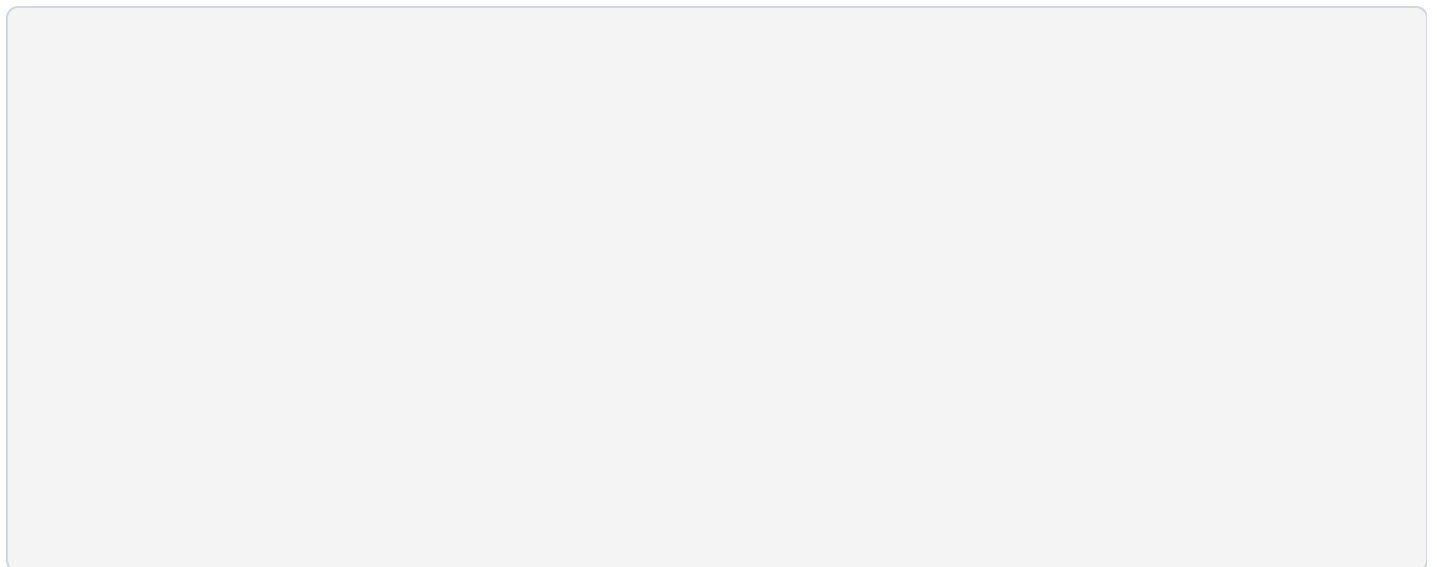
1. La View non parla mai al DB. Chiede al Controller, che chiede al DAO.
2. Il DAO non conosce il dominio applicativo. Riceve e ritorna oggetti, esegue query, basta.
3. Il Controller orchestra. Conosce sia il DAO che gli oggetti applicativi (SessionContext, ecc.), può gestire transazioni cross-DAO.
4. Le query SQL stanno in **QueryContainer** , non sparse nei DAO.
5. I DTO sono read-only, usati quando un'operazione restituisce dati joinati da più tabelle.

3. Stack tecnologico

Tecnologia	Versione	Uso
Java	8+	Linguaggio principale
Swing	JDK	UI desktop (<code>JFrame</code> , <code>JTable</code> , <code>JComboBox</code> , ecc.)
NetBeans Form Editor	12+	Generazione codice UI (<code>.form</code> ↔ <code> initComponents()</code>)
MySQL	5.7 / 8.x	Database relazionale
JDBC	mysql-connector-j	Accesso DB
Ant	NetBeans	Build system (<code>build.xml</code>)

Niente framework esterni : il progetto è didattico, deve usare solo JDBC puro + Swing puro per esibire la conoscenza delle API standard. Niente Spring, Hibernate, Lombok.

4. Struttura del progetto



4.1 Package controllers

LoginController, HomeController, RegistrationController, PasswordResetController, ManageUserController, ManageTenantController, ManageRoomController, ReservationController, CheckInController, CheckOutController, FatturaController, UserController.

4.2 Package dao

DatabaseConnection (singleton), UserDao, IndirizzoDao, MansioneDao, CommittenteDao, RoomDao, ReservationDao, FatturaDao, MetodoPagamentoDao, PagamentoDao, AccessLogDao, SessionsDao, StoricoPrenotazioniDao.

4.3 Package dto

LoginResponse, FatturaDettaglio, ReservationDettaglio, CommittenteDettaglio, CheckoutPreview, AccessLogDettaglio.

4.4 Package enums

UserStatus, Genere, RoomType, RoomStatus, BedType (*smart enum*), ReservationStatus, FatturaStatus (*smart enum*).

4.5 Package models

User, Indirizzo, Mansione, Committente, Room, Reservation, Fattura, MetodoPagamento, Pagamento, AccessLog, Sessions, StoricoPrenotazioni.

4.6 Package views

LoginView, LoadingView, Loading1View, RegistrationView, PassWordView, HomeView, ManageRiservationView, ManageRoomView, ManageUserView, ManageTenantView, CheckInView, CheckOutView, FatturaView, GestioneFattureView, LogsView.

5. Database

5.1 Schema sintetico

```
Indirizzo (indirizzo_id, paese, provincia, citta, via, cap)
Mansione (role_id, role_type, role_nome)
Committente (codCommittente, ragione_Sociale, gestore→User, email, telefono, indiriz
User (user_counter VARCHAR PK, nome, cognome, email, access [password],
    ruolo→Mansione, committente_id→Committente,
    stato ENUM('attivo', 'disattivato', 'attesa'),
    telefono, indirizzo_id, recupero, response, genere)
Room (room_id, committente_id, numero_stanza,
    tipo ENUM('singola', 'doppia', 'suite'), prezzo,
    letto_tipo ENUM('matrimoniale', 'singolo', 'king-size'),
    stato ENUM('disponibile', 'occupata', 'manutenzione'))
Reservation (reservation_id, user_id, committente_id, room_id,
    check_in DATE, check_out DATE, giorni INT,
    status ENUM('attiva', 'cancellata', 'completata'), note)
Fattura (fattura_id, reservation_id, importo, data_emissione DATETIME,
    stato ENUM('pagato', 'non pagato', 'in attesa'))
MetodoPagamento (metodo_id, nome, descrizione)
Pagamento (pagamento_id, fattura_id, metodo_id, importo, data_pagamento)
AccessLog (log_id AUTO_INCREMENT, user_id, ip_address, note,
    login_time DEFAULT NOW(), logout_time NULL)
sessions (session_id, user_id, committente_id, role_id,
    token CHAR(64), expires DATETIME, created_at)
StoricoPrenotazioni (storico_id, reservation_id, user_id,
    check_in_precedente, check_out_precedente,
    nuovo_check_in, nuovo_check_out, data_modifica)
```

Note importanti :

- `User.user_counter` è `VARCHAR(100)` , non int autoincrementale. Generazione applicativa via `UserDao.generateUniqueCounter()` con pattern `"XXX{id}{NOME}"` .
- `Fattura.stato` ha valori con spazi (`"non pagato"` , `"in attesa"`), gestiti via smart-enum `FatturaStatus` .
- `Room.letto_tipo` ha il trattino (`"king-size"`), gestito via smart-enum `BedType` .
- `Reservation.check_in/check_out` sono `DATE` (giorno solo, non timestamp).

5.2 Stato Models / DAO

Tabella DB	Model	DAO
Indirizzo	✓	✓
Mansione	✓	✓
Committente	✓	✓
User	✓	✓
Room	✓	✓
Reservation	✓	✓
Fattura	✓	✓
MetodoPagamento	✓	✓
Pagamento	✓	✓
AccessLog	✓	✓
sessions	✓	✓
StoricoPrenotazioni	✓	✓

5.3 Configurazione connessione

I parametri di connessione vivono in `src/db.properties` (non versionato per sicurezza) :

```
db.url=jdbc:mysql://localhost:3306/pm_collegio
db.user=root
db.password=YOUR_PASSWORD
```

6. Layer applicativi

6.1 Models

POJO che rappresentano 1:1 una tabella del DB. Solo dati + getter/setter, niente logica. Esempio :

```
public class Room {
    private int id;
    private int id_committente;
    private int numeroStanza;
    private double prezzo;
    private RoomType rTipo;
    private BedType lettoTipo;
    private RoomStatus stato;
    // getter / setter / costruttori
}
```

6.2 Enums

Due categorie :

Enum semplici (match diretto via uppercase/lowercase)

- `UserStatus` : `ATTIVO` , `DISATTIVATO` , `ATTESA`
- `Genere` : `MASCHIO` , `FEMMINA` , `ALTRO`
- `RoomType` : `SINGOLA` , `DOPPIA` , `SUITE`
- `RoomStatus` : `DISPONIBILE` , `OCCUPATA` , `MANUTENZIONE`
- `ReservationStatus` : `ATTIVA` , `CANCELLATA` , `COMPLETATA`

Smart enum (campo `dbValue` esplicito, per valori DB con spazi/trattini)

```
public enum FatturaStatus {
    PAGATA("pagato"),
    NON_PAGATA("non pagato"),
    IN_ATTESA("in attesa");

    private final String dbValue;
    FatturaStatus(String dbValue) { this.dbValue = dbValue; }

    public String getDbValue() { return dbValue; }
    public static FatturaStatus fromDb(String dbValue) { ... }
}
```

Stesso pattern per `BedType` (`"king-size"` con trattino).

6.3 DTO

Oggetti di **trasporto** per dati joinati o calcolati. Read-only nel senso applicativo : non vanno mai serializzati al DB direttamente.

DTO	Scopo
<code>LoginResponse</code>	Esito login : successful, user, errorMessage. Factory <code>ok()</code> / <code>error()</code>
<code>FatturaDettaglio</code>	Vista joinata Fattura + Cliente + Camera per GestioneFatture
<code>ReservationDettaglio</code>	Vista joinata Reservation + User.nome + Room + totale calcolato
<code>CommittenteDettaglio</code>	Committente + email del gestore + via dell'indirizzo
<code>CheckoutPreview</code>	Dati pre-calcolati per la schermata CheckOut (giorni, totale)
<code>AccessLogDettaglio</code>	Log + nome utente + ruolo nome (per LogsView)

6.4 DAO

Una classe per ogni tabella del DB. Espongono i metodi CRUD :

- `CheckInController.prenota( ... )` : transazione con `setAutoCommit(false)` → insert Reservation → insert Fattura (importo 0) → update Room=OCCUPATA → commit (o rollback in caso di errore).
- `CheckOutController.eseguiCheckout( ... )` : *ricalcola server-side* importo e giorni (no fiducia nei dati view), poi transazione UPDATE Reservation + UPDATE Room + upsert Fattura.
- `ReservationController.aggiornaConStorico( ... )` : UPDATE Reservation + INSERT StoricoPrenotazioni (audit con vecchie date) in transazione.
- `UserDao.generateUniqueCounter(nome)` : conta utenti esistenti, prova `XXX{count+1}{NOME}` , se collide incrementa fino a max 100 retry.

6.6 Views

`JFrame` Swing. Ogni view :

- Si costruisce con un controller dedicato come campo `final`
- Chiama `initComponents()` (autogenerato da NetBeans Form Editor) per i componenti UI
- Aggancia action listener custom (programmatici) che chiamano metodi del controller
- Riceve Model o DTO dal controller e li riversa nei componenti UI
- Non esegue mai query SQL

Pattern UI ricorrenti

- Validazione live (key released su email/telefono) → label con feedback  / 
- Add/Update mutex : click su una riga della tabella → Add disabilitato + Update abilitato. Sempre presente in ManageUser, ManageTenant, ManageRoom.
- `configureForRole()` : per ruolo U nasconde/disabilita elementi non permessi.

6.7 Utilities

Classe	Scopo
<code>utility</code>	Helper statici : <code>convAlfaR(int)→"R0042"</code> , <code>convAlfaP(int)→"P0042"</code> , <code>convInt(String)→42</code> , <code>isValidEmail</code> , <code>validaNumeroTelefono</code> , <code>calculDate(d1, d2)</code>
<code>QueryContainer</code>	Tutte le query SQL del progetto come <code>public static String</code> . Centro unico per modificarle.
<code>SessionContext</code>	Stato globale dell'utente loggato (campi <code>static</code> : <code>userCounter</code> , <code>email</code> , <code>nome</code> , <code>cognome</code> , <code>roleType</code> , <code>sessionToken</code> , <code>logId</code> , ecc.). <code>clear()</code> al logout, <code>isLoggedIn()</code> come check.

Attenzione : `SessionContext` (utility, stato in memoria) è diverso da `Sessions` (model della tabella `sessions` nel DB). Naming volutamente differente per evitare ambiguità.

7. Flusso applicativo end-to-end

Scenario : utente cliente prenota una camera e paga la fattura.

8. Pattern e principi applicati

8.1 Principi SOLID

Principio	Applicazione in v2
S — Single Responsibility	Ogni DAO una sola tabella. <code>UserDao</code> gestisce solo <code>User</code> , <code>AccessLogDao</code> solo <code>AccessLog</code> . Niente God object.
O — Open/Closed	Aggiungere un nuovo stato fattura (es. <code>RIMBORSATA</code>) richiede solo di estendere l'enum <code>FatturaStatus</code> . Niente modifiche al DAO.
L — Liskov Substitution	Non particolarmente esposto (poche gerarchie), ma rispettato dove c'è.
I — Interface Segregation	Niente interfacce in v2 (semplicità). In un sistema reale i DAO esporrebbero <code>IXxxDao</code> .
D — Dependency Inversion	Parzialmente. I controller dipendono dai DAO concreti. Con DI vera (Spring / .NET) sarebbe meglio.

8.2 Pattern strutturali

- MVC esteso a MVC + DAO + DTO
- Singleton : `DatabaseConnection.getConnection()` ritorna sempre la stessa istanza. Essenziale per transazioni cross-DAO.
- Factory Method : `LoginResponse.ok(user)` / `LoginResponse.error(msg)` per costruire DTO in modo leggibile.

8.3

```
... } }
```

```
... / rollback() →
```

8.4

`checkout` non si fida dei campi della view per

`controller.cercaDettaglio` aggiunge `AND`

`passwordResetController.resetPassword` ritorna sempre lo

la registrazione crea utenti che non possono ancora loggare.
admin.

8.5

ordinate absolute con spacing costante : in HomeView gap = 115 pixel tra bottoni.

9. Sicurezza e validazioni

9.1 Validazioni client-side (View)

- Email : regex `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6}$` in `utility.isValidEmail` . Feedback live su `keyReleased` .
- Telefono : esattamente 10 cifre in `utility.validaNumeroTelefono` . Feedback live.
- Campi obbligatori : check `isEmpty()` prima di chiamare il controller.
- Conferme distruttive : `JOptionPane.showConfirmDialog` prima di Delete / Logout / Annulla fattura.

9.2 Validazioni server-side (Controller)

- Checkout : importo ricalcolato dal DB, non dal form
- Cambio prenotazione : registra audit in `StoricoPrenotazioni`
- Filtro ruolo U : sempre lato server, anche se la UI già limita
- `existsByEmail` / `existsByCounter` : nel `UserDao` prima di INSERT per evitare duplicati
- `isOccupata` : `ManageRoomController` blocca delete/update di camere occupate

9.3 Limiti di sicurezza

- Password in chiaro nel DB (campo `User.access`).
- Niente rate limiting : su login e reset password un attaccante può provare migliaia di volte.
- Domanda di sicurezza è notoriamente debole

10. Setup ed esecuzione

10.1 Prerequisiti

- JDK 8+
- NetBeans 12+ (con Ant)
- MySQL 5.7+ (server in esecuzione su `localhost:3306`)
- Connector `mysql-connector-j` nelle library NetBeans

10.2 Setup database

1. Aprire MySQL Workbench (o riga di comando)
2. Creare schema : `CREATE DATABASE pm_collegio;`
3. Eseguire lo script SQL completo (incluso in `SpecificaTecnica.docx`) per creare tutte le tabelle + seed
4. Verificare che ci sia almeno un utente con `stato='attivo'` per fare login

10.3 Configurazione connessione

In `src/db.properties` (creare se non esiste) :

```
db.url=jdbc:mysql://localhost:3306/pm_collegio
db.user=root
db.password=YOUR_PASSWORD
```

10.4 Esecuzione

1. Aprire il progetto in NetBeans (`File` → `Open Project` → `PM_Collegiov2`)
2. Verificare che `mysql-connector-j` sia nelle Libraries del progetto
3. Click destro sul progetto → `Clean and Build`
4. Click destro sul progetto → `Run` (o F6)

5. La main class è `LoadingView` (splash + progress) → si apre automaticamente `LoginView`